



COPILOTTRAINING
TECHNICAL BRIEFING

Multi-Agent Coordination

| Split, Steer, and Integrate Isolated Workstreams

Create parallel value through explicit contracts and named integration ownership

CopilotTraining Tech Talk - 55 minutes - Advanced



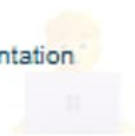
"When should work be split across agents, and how should isolated workstreams be integrated?"

Concurrency earns its overhead only when boundaries are stable and combined behavior is observable.

Choose the shape, contract the work, then reunify it under one owner.

Developer

Choose a split that keeps implementation reviewable.



Team Lead

Assign ownership, evidence, and escalation before dispatch.



Architect

Preserve system contracts across concurrent workstreams.



2 Variables

Coupling plus integration risk determines the coordination shape.

3 Hosts

One contract briefs subagents, background agents, and Copilot App sessions.

7 ≠ 14

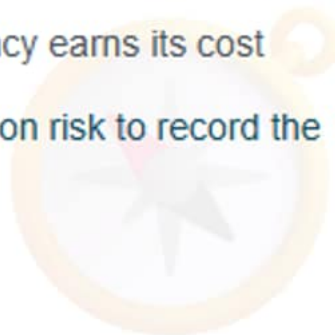
Integration exposes a client assumption against the accepted contract.

PART 1

Choose a Coordination Shape

Decide whether concurrency earns its cost

Use coupling and integration risk to record the split decision.



PART 2

Define Roles and Authority

Turn the split into a portable contract

Bound scope, evidence, autonomy, and escalation before dispatch.

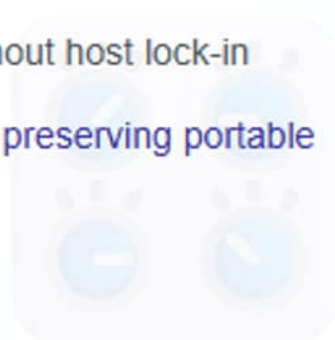


PART 3

Isolate and Steer Workstreams

Operate a visible fleet without host lock-in

Use the Copilot App while preserving portable workstream contracts.

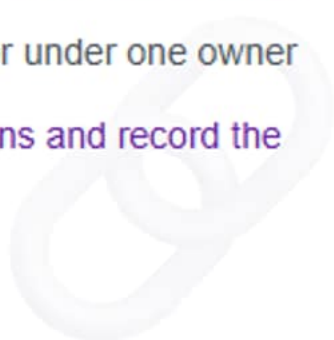


PART 4

Integrate Results Deliberately

Validate combined behavior under one owner

Catch divergent assumptions and record the acceptance decision.



Choose a Coordination Shape

Use coupling and integration risk to select and record a defensible split.



Coupling

Test whether inputs can stabilize first



Integration Risk

Price the cost of recombining outputs



Recorded Decision

Make the topology reviewable

Coordination decision

↳ 1 contract decision → 4 concurrent workstreams

Four Shapes, Chosen by Coupling and Integration Risk

Select the least complex topology that preserves reviewable evidence

Single stream	One coherent change with one obvious validation path	high coupling
Sequential	Validated artifacts pass forward when dependencies are unresolved	handoff gates
Parallel	Stable inputs and disjoint scopes enable concurrent execution	low coupling
Deliberation	Independent perspectives expose risk before commitment	high consequence

Every workstream must earn its dispatch, review, and merge overhead



Can the Work Separate and Reunify Cleanly?



Task Coupling

Stable input

A stream can proceed without changing upstream output

Separate scope

Concurrent writers do not share mutable files or state

Reviewable output

The artifact can be judged independently



Integration Risk

Contract sensitivity

Shared interfaces can drift across streams

Combined proof

Integrated behavior needs an observable check

Acceptance owner

One authority can resolve and accept the result

coordination-shape.md Makes the Split Reviewable

coordination-shape.md

Coordination Shape

Candidate task

Replace the session-token format.

Coupling check

Contract decision precedes all writers.

Selected shape

1. Sequential: approve token contract
2. Parallel: server, client, threat review, migration tests
3. Sequential: integrate and validate

Split test

Stable input, separate scope, reviewable output, named evidence



Freeze the Contract

Resolve the token decision before concurrent implementation.



Split by Dependency

Give each stream a stable input and disjoint writable scope.

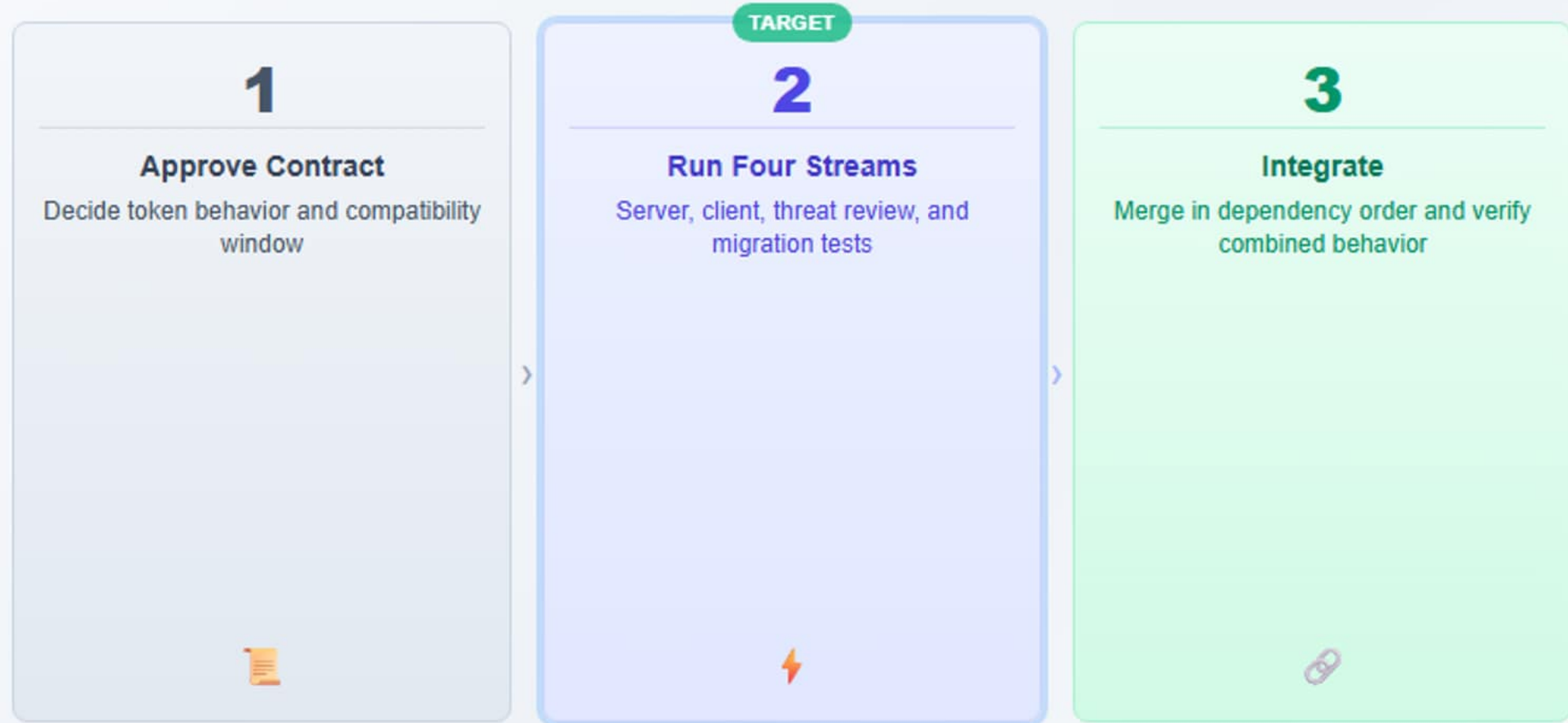


Name the Proof

Record evidence and the acceptance owner before dispatch.

One Sequential Decision Unlocks Four Concurrent Streams

Parallelism begins only after the shared token contract stabilizes



A multi-agent plan can preserve sequencing where dependency demands it

Define Roles, Handoffs, and Authority

Carry scope, evidence, autonomy, and escalation in one portable contract.



Bounded Objective

Name the output and writable scope



Evidence

Define acceptance before execution



Authority

Separate execution from approval

```
coordination-contract.yml  
↳ One brief · three hosts · named acceptance owner
```

One Workstream Block Carries the Decisive Context

coordination-contract.yml

```
- id: client
  role: implementer
  dependsOn: [contract-approved]
  writableScope:
    - src/client/**
    - tests/client/**
  output: Client adoption of token v2
  evidence: npm test -- client
  autonomy: plan
  escalateWhen: >
    Backward compatibility cannot be preserved
```

Objective and Input

The accepted contract removes hidden context.

Writable Scope

Paths make ownership and collision risk explicit.

Evidence and Stop

A command proves the return; escalation bounds autonomy.

A Role Label Is Not a Coordination Contract



Objective

Name the output, decisive inputs, and explicit non-goals.



Scope

Bound writable paths, tools, runtime, and shared state.



Evidence

Require a test, artifact, or finding tied to acceptance.



Escalation

Stop when assumptions, public contracts, or compatibility change.

Match Session Autonomy to Workstream Stability



Interactive

Explore when scope or approach can change.

Approve consequential decisions as they appear



Plan

Review design before stable implementation begins.

Approve the plan before execution



Autopilot

Run routine work with mechanical validation.

Review the returned artifact and evidence

Session Mode Does Not Grant Merge Authority



Agent Execution Authority

Propose a plan within the objective

Implement inside the writable scope

Run named validation commands

Report evidence and changed assumptions



Human Acceptance Authority

Redirect or reject a workstream

Resolve cross-stream conflicts

Accept combined behavior

Merge through repository policy



Branch protection and named reviewers retain organizational authority.

Isolate and Steer Workstreams

Use sufficient isolation and fleet state to make timely operator decisions.



Isolation

Match the boundary to collision risk



Fleet State

Observe work before intervening



Steering

Act on explicit decision triggers

GitHub Copilot App fleet

↳ Visible operation · portable coordination contract

Isolation Reduces Interference, Not Integration Work

Add layers according to the collision and consequence risk

Context

Subagents protect focus; shared files can still collide

subagent

Tools

Custom roles bound actions; allowed tools still affect state

custom agent

Files

Writable scopes separate edits; shared contracts can drift

path scope

Worktree

Branches isolate directories; merge conflicts remain

background

Runtime

Sandboxes isolate processes; environments can diverge

sandbox

Choose the lowest layer that controls the actual risk

Six Questions Turn Fleet State into Operator Decisions

The GitHub Copilot App is the visible surface; the questions remain portable

Ownership	Which workstream owns this output?	assign
------------------	------------------------------------	---------------

State	Is it planned, running, blocked, ready, or rejected?	observe
--------------	--	----------------

Assumption	Which accepted premise changed?	compare
-------------------	---------------------------------	----------------

Evidence	What artifact or validation has arrived?	inspect
-----------------	--	----------------

Intervention	Which stream needs human input now?	steer
---------------------	-------------------------------------	--------------

Dependency	What must arrive before integration advances?	sequence
-------------------	---	-----------------

The same contract steers subagents, background agents, and worktrees

Integrate Results Deliberately

Reunify returned evidence, resolve divergence, and record acceptance under one owner.



Returned Evidence

Inspect outputs before combining



Contract Check

Expose assumptions at the boundary



Acceptance

Record proof and the owner's decision

```
integration-record.md
└ 7-day assumption ≠ 14-day accepted contract
```

Three Green Workstreams Still Need One Consistency Check



Server · PR #412

Dual-read support for token v1 and v2.

npm test -- auth passes

Status: ready



Client · PR #415

Client writer adopts token v2.

npm test -- client passes

Status: ready



Threat Review

Findings mapped to the accepted contract.

0 blockers, 2 accepted notes

Status: ready

Locally Green, Contractually Inconsistent

The integration record compares assumptions before merge

7 ≠ 14

days in the client assumption versus the accepted contract

Source: *integration-record.md*

Accepted Contract

Compatibility window: fourteen days.

Client Return

A constant encoded a seven-day window.

Why Tests Passed

Scoped client tests validated local behavior, not shared intent.

⚠ Integration caught a semantic conflict that isolated evidence could not reveal.

integration-record.md Turns Divergence into a Decision

integration-record.md

Conflict resolution

- Client assumed a seven-day compatibility window.
- Accepted contract requires fourteen days.
- Resolution: update the client constant.
- Rerun client tests before merge.

Integration order

1. Merge server dual-read support.
2. Merge corrected client writer.
3. Run combined suite and migration smoke test.

Decision

Accepted by platform-lead.

Compare

Check every return against the same accepted contract.

Correct

Update the client constant from seven to fourteen days.

Rerun

Repeat scoped and combined checks on the corrected state.

One Named Integrator Owns Coherence

Acceptance depends on the integrated state and a recorded authority decision

Complete

Every required artifact and evidence item arrived

all returns

Consistent

All streams used the same contracts and assumptions

7 → 14 days

Compatible

Interfaces and migrations compose in dependency order

ordered merge

Combined

Suite and migration smoke test pass after integration

integrated proof

Authority

The platform lead records accept, redirect, or reject

human decision

Failed integration improves the next split decision: reduce overlap or move shared decisions earlier



From Parallel Activity to Coordinated Delivery

Before

Split work because parallelism looks faster

Let each session infer its own boundaries

Track activity without decision triggers

Discover contract drift during merge

After

Use coupling and integration risk to choose the shape

Dispatch one portable coordination contract

Steer visible fleet state through named authority

Validate combined behavior in an integration record

1 → 4

Contract decision unlocks concurrent workstreams

3 hosts

One portable workstream contract

7 ≠ 14

Divergent assumptions caught before merge

✓ WHAT YOU CAN DO TODAY

Today

- Try: score one backlog item for coupling and integration risk
- Signal: record one topology and why each stream can proceed
- Validate: ask a reviewer to find unresolved dependencies

This Week

- Build: contract two read-only specialist workstreams
- Signal: each returns its artifact without hidden cross-context
- Validate: map every accepted claim to returned evidence

This Month

- Pilot: dispatch two low-risk isolated implementation streams
- Measure: scope adherence, local proof, and integration effort
- Pause: return to sequential work when shared assumptions drift

🔑 Key Takeaway

Parallel work creates value only when its contracts and combined behavior remain observable.

 Agent Operating Surfaces[Subagents in VS Code](#)

Isolated context, parallel delegation, and result return.

[Custom agents in VS Code](#)

Focused roles, tools, invocation, and handoffs.

[Background agents in VS Code](#)

Asynchronous execution in isolated workspaces.

[Git worktree reference](#)

Independent working trees sharing one repository.

 Fleet Operation[About the GitHub Copilot app](#)

Multi-session work, operating controls, and review surfaces.

[Getting started with the Copilot app](#)

Current setup and first-session workflow.

[VS Code agent orchestration](#)

Orchestration and session-management release baseline.



Multi-Agent Coordination

SPLIT, STEER, AND INTEGRATE ISOLATED WORKSTREAMS

Choose

Use coupling and integration risk before dispatch.

Contract

Carry scope, evidence, autonomy, and escalation across hosts.

Integrate

Let one owner validate combined behavior and accept the result.

Where would a concurrent split improve your work, and what evidence would make it safe to reunify?